Applying Order Based Scheduling to Flatland

David Bunker[1] and Hannes Weichelt[1]

University of Potsdam, Potsdam, Germany
https://uni-potsdam.de

Abstract. Train scheduling is a challenging problem with many possible methods, including reinforcement learning, constraint programming, and hybrid approaches. This problem becomes exponentially more difficult as more trains, tracks, and junctions are added. By focusing on ordering, complexity is reduced from needing to represent every time step. This work applies this approach to the 2020 Flatland competition using Answer Set Programming for orderings to get the final train movements. The tools used in development, Flatdraw, Visualizer and others are also discussed.

Keywords: Flatland · Scheduling · Answer-Set Programming.

1 Introduction

Although many strides have been made in recent years on train scheduling technology, it remains challenging [2]. As the number of trains and number of track junctures increase, the number of possible routes increases exponentially and finding a route for all trains that avoids collisions has been demonstrated to be NP-Complete [8]. Given its NP-Complete nature, constraint programming continues to be an effective choice for tackling such scheduling problems [1] [7]. In this work The constraint programming language Answer Set Programming (ASP) was applied to the Flatland AI challenge [5].

2 Motivation and Related Work

The goal of the Flatland Challenge is to find non-conflicting train paths in an abstract gird environment that emulates a real world rail network. Typical ASP encodings for solving this challenge usually have scaling problems due to increasing grounding times resulting from having to ground train positions over all nodes of the environment for each time step. From these, time is often the deciding factor which makes grounding instances with high horizons very inefficient.

This problem can be avoided by removing the time steps from the instances by instead generating a partial ordering over the junction nodes of the environment. Once this ordering is generated, the time steps can be later generated in a post-processing step.

The motivation for this approach is to reduce the amount of grounding for larger instances that have many trains. A similar approach for Multi Agent Path-Finding (MAPF) has been demonstrated to be effective [4]. Since MAPF is a problem related to Flatland, this approach is based on these methods [4] and adapted to Flatland. Further optimizations were performed by introducing a condensed graph representation to further reduce the grounding space.

3 Background

3.1 From Grid to condensed Graph

In flatland, the modeled maps are represented by a set of tracks that are distributed in a grid formation where neighboring cells are connected to each other depending on their type of track. The problem of finding paths for trains on this map can be translated to a graph problem. Here, each directed cell is converted to a node in the graph representation where the track connections between cells are translated to the edges between these directed cells. The difference between a normal cell and a directed cell is that for directed cells the direction of the incoming train into the cell is added to the node information since the outgoing edges vary depending on this direction.

3.1.1 Normal Graph Representation

The graph representation of this map $G = (N, E)$ then features the nodes $N = \{node((x, y), dir) | cell(y, x, t), direction(dir)\}$ and edges $E = \{edge(n_1, n_2) | n_1, n_2 \in N, n_1 \text{ and } n_2 \text{ are connected}\}$. Which edges are valid connections and which are not can be determined through the two track ids t_1, t_2 in $cell(x_1, y_1, t_1)$ and $cell(x_2, y_2, t_2)$ for both nodes in $edge(node((x_1, y_1), dir_1), node((x_2, y_2), dir_2))$. Here, the track IDs binary representation actually represents the valid traversal options over this node. More information about this process can be found in [6]. Now, with the two directions dir_1, dir_2 the valid connections between these nodes can be determined if there are any. This process is illustrated in Figure 1 below.

3.1.2 Condensed Graph Representation

However, this graph representation has straight nodes in the track which do not leave the trains on them any choice but to move forward. This means when searching for a conflict-free schedule for all trains these nodes cannot play a role and thus can be left out of that decision process. Thus, The concept of the condensed graph G_c is introduced where these intermediary nodes are removed and replaced by weighted edges accordingly. Solving over this condensed graph is beneficial as it features fewer nodes and edges than G and thus reduces the amount of grounding needed to be done. G_c is defined as $G_c = (N_c, E_c, W)$ where $N_c \subseteq N, E_c \subseteq E$, and $W = \{weight(e, w) | e \in E_c, w \in \mathbb{N}\}$. Figure 2 shows the example graph from Figure 1 compared to its condensed version.

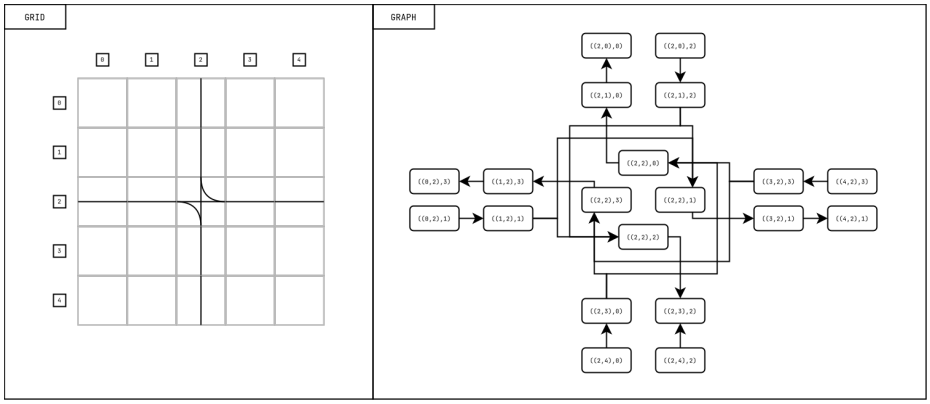


Fig. 1: Transformation for grid to graph representation

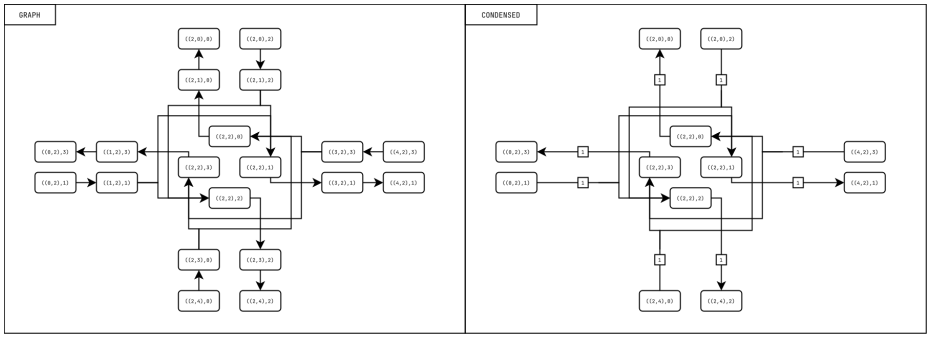


Fig. 2: Transformation for graph to condensed representation

It is important to note that in figure 2 the end nodes `(2,0,0)`, `(2,0,2)`, `(4,2,1)`, `(4,2,3)`, `(2,4,0)`, `(2,4,2)`, `(0,2,1)`, `(0,2,3)` are conserved here for the sake of illustration. In the actual implementation these nodes would only be conserved if a train had its start or goal position on them.

3.2 Partial Train Ordering

The concept of assigning a partial train ordering to the relevant switch nodes of our map is inspired by the approach taken in [4] for MAPF. One of the major benefits of this approach is that time can be disregarded for computing this partial ordering, thus greatly reducing the grounding space needed. Though it is important to mention that due to this the resulting plans also disregard the assigned time horizons and earliest starting times. A number of methods to adapt these time constraints to the partially ordered solutions were attempted, however an improved and more efficient method over the prototype was not determined. These approaches are described in 8 (Future Work).

4 Approach

The Flatland Challenge was addressed using both a baseline action-based approach in addition to the ordering-based approach to provide an encoding for comparison.

4.1 Baseline Implementation

The baseline encoding implementation[1] follows an action based approach similar to that taken by the `krr-up/flatland`[2] repository.

This encoding uses the transitional edges of the Flatland grid to choose a set of taken actions over these edges such that all trains reach their destination without conflicts. Here, for each time step an action is chosen for each train. This is where the grounding space rapidly increases with the increasing time horizon. The details of the implementation can be found in the provided GitHub repository.

4.2 Partial Ordering Implementation

The partial ordering implementation consists of a number of steps implemented as separate encodings before being combined and run on instances followed by a post-processing step. The first step is to convert the track types to edges between the cells as outlined in Section 3.1.1, to make it simple to define train movement.

Once this is done this graph can be simplified to consist of only transitions between track switches where a decision must be made (outlined in Section 3.1.2). To complete the encoding, the `move` atom is introduced to indicate train movement between switches and the `resolve` atom to indicate which order they must enter.

The final post-processing step converts the movement of the agents and the ordering at the switches to form the action steps each train takes.

4.2.1 Track to Edge Representation

Tracks are represented using a 16-bit number consisting of a from and to direction. The first bit is NN or north to north, which is in the representation, but can never happen, followed by NE for north to east, etc. This can be made explicit by the following encoding.

```
bits((NN,NE,NS,NW),(EN,EE,ES,EW),(SN,SE,SS,SW),(WN,WE,WS,WW), T) :-
    track_type(T),
    WW=(T/(2**0))\2,  WS=(T/(2**1))\2,  WE=(T/(2**2))\2,  WN=(T/(2**3))\2,
    SW=(T/(2**4))\2,  SS=(T/(2**5))\2,  SE=(T/(2**6))\2,  SN=(T/(2**7))\2,
    EW=(T/(2**8))\2,  ES=(T/(2**9))\2,  EE=(T/(2**10))\2, EN=(T/(2**11))\2,
    NW=(T/(2**12))\2, NS=(T/(2**13))\2, NE=(T/(2**14))\2, NN=(T/(2**15))\2.
```

[1] https://github.com/dbunker/railway/blob/main/encodings/rail$_n ew_a ctions.lp$

[2] https://github.com/krr-up/flatland

From this we can derive `transitions` representing all possible transitions such as left to right and right to left. This can be used to get possible actions associated with a track and a given direction, such as `move_left` or `wait`, which is always possible. From this we can link cells together with edges of the form stated below.

```
edge((X_BEFORE, Y_BEFORE), DIRECTION_BEFORE,
    (X_AFTER, Y_AFTER), DIRECTION_AFTER, ACTION_PERFORMED).
```

4.2.2 Consolidation to a Simplified Graph

Once this graph between cells is created it can be simplified as only the places where an action can be performed are relevant. Although at any point on a track with no directions a train can wait, the track will be blocked in the opposite direction for the same waiting time regardless. If the train is to wait it should always wait immediately before the switch. This will be represented later in the ordering encoding which defines the order by which trains must occupy each switch.

A switch is any cell with at least 3 exiting directions. All directions of switches are also switches even if there are not multiple paths.

```
switch(N) :- initial_node(N), 2 {initial_edge(N,_)}.
switch(((X,Y), DIR2)) :- switch(((X,Y), DIR)),
    initial_node(((X,Y), DIR2)), DIR!=DIR2.
```

Next, all the ways in which switches are reachable between each other are defined. As seen below the non-switch node `N` is reachable from node `SWITCH_1` if it is connected by an edge. This extends out from this reachable point.

```
reachable(SWITCH_1, N, 0) :- initial_edge(SWITCH_1, N),
    switch(SWITCH_1), not switch(N).
reachable(SWITCH_1, N2, D+1) :- reachable(SWITCH_1, N1, D),
    initial_edge(N1, N2), not switch(N2).
```

A connection exists when two switches are reachable by way of a reachable chain or next to each other.

```
connection(SWITCH_1, SWITCH_2, 0) :- initial_edge(SWITCH_1, SWITCH_2),
    switch(SWITCH_1), switch(SWITCH_2).
connection(SWITCH_1, SWITCH_2, D+1) :- reachable(SWITCH_1, N, D),
    initial_edge(N, SWITCH_2), switch(SWITCH_2).
```

The starting and end positions of all trains should also be represented as switches. The switches are now the new nodes and connections are the new edges in the simplified graph. These can be represented with Clingraph to help in debugging as shown in Section 5.2.

4.2.3 Ordering Encoding

The ordering encoding generates `resolve(ID1, ID2, SWITCH)` meaning that `ID1` came before `ID2` at `SWITCH` and `move(ID, PREVIOUS_SWITCH, NEXT_SWITCH)` meaning that move happened from which train actions can be produced. This reduces the potential atoms that could exist via grounding by a factor up to the max iteration time. First, the path candidates must be defined. Atom `move(ID, U, V)` can exist once among possible edges.

```
{ move(ID, U, V) : output_edge(U, V, _) } <= 1 :- train(ID), output_node(V).
{ move(ID, U, V) : output_edge(U, V, _) } <= 1 :- train(ID), output_node(U).
```

Next, move transitivity constraint ensures that moves can only occur in one direction and never reverse. A move must also come from the spawn and end at the goal.

Resolve occurs at spawn and goal if there are any other trains that enter them. If two trains enter the same node, exactly one must resolve before the other as shown below also using symmetry splitting.

```
{ resolve(ID1, ID2, (X,Y)); resolve(ID2, ID1, (X,Y)) } >= 1 :-
    move(ID1, _, ((X,Y) ,_)), move(ID2, _, ((X,Y), _)), ID1 < ID2.
:- resolve(ID1, ID2, (X,Y)), resolve(ID2, ID1, (X,Y)).
```

Now to define order waited based on `move` and `resolve`. One train must come before the other.

```
order_waited(T, (X,Y), T2) :- move(T, _, ((X,Y) ,_)), resolve(T2, T, (X,Y)).
order_waited(T, N, T3) :- order_waited(T,N,T2), resolve(T3, T2, N).
:- order_waited(T1, N1, T2), order_waited(T2, N2, T1).
```

Finally, prohibit move and resolve cycles.

```
#edge ((ID,U), (ID,V)) : move(ID, U, V).
#edge ((ID1,V), (ID2,U)) : resolve(ID1, ID2, U), move(IDA, U, V).
```

4.2.4 Post-Processing

The final post-processing step takes the `resolve` and `move` atoms and creates `action(train(ID), TIMESTEP, ACTION)` where `ACTION` is move left, right or wait. This is done by always moving trains forward unless a switch is reached, at which point the train waits for its turn before continuing forward along its defined movements. However, this approach is not guaranteed to meet the earliest departure and latest arrival times, but if this constraint can be met and does not involve a cycle, one of the `resolve` and `move` answer sets will meet it. Although this is generally a large number of answer sets that must be checked to find the optimal one.

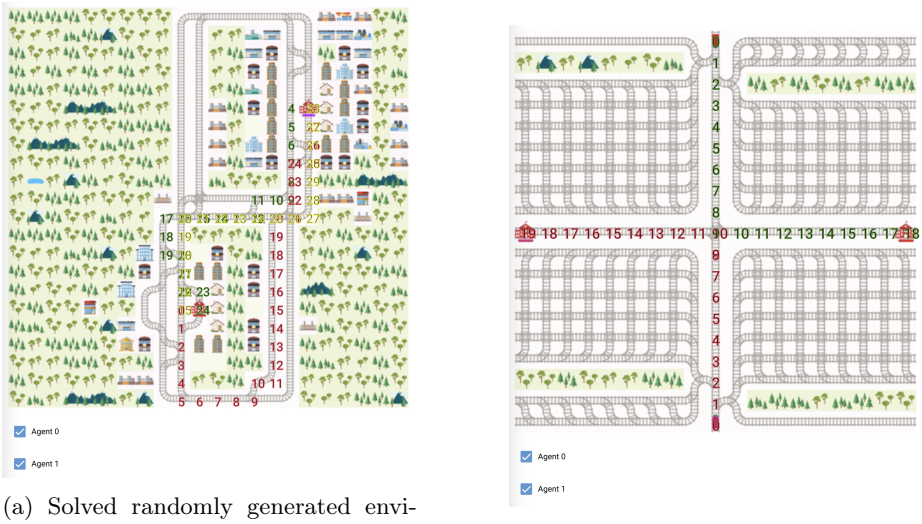
The last post-processing step is implemented in Python to interpret the ordering drawn from the order ASP encoding. As there is no more path-finding to be done, but only executing the already computed plan, this is an efficient route.

5 Additional Tools

During work on this project, some additional tools were also developed to aid in development. This section describes these tools and the application of them.

5.1 Visualizer

The Visualizer tool as shown in Figure 3 enables the user to specify an `instance.lp` containing `train`, `cell`, `start`, and `end` atoms to solve or randomly generates an environment to solve using the Flatland library. The paths of each train along with the timestamp associated with the movement are shown overlapped over the flatland environment background. The `NiceUI` web framework used allows for the user to select individual train paths on or off for better visibility. This tool, particularly when combined with Flatdraw (Section 5.3), make understanding and debugging Flatland encodings much easier.



(a) Solved randomly generated environment

(b) Solved user defined environment

Fig. 3: Multiple ways to use Visualizer

5.2 Condensed Graph Visualization

To visualize the condensed graph as seen in Section 3.1.2 a visualization encoding[3] using `clingraph` [3] was created. This is useful in visualizing the condensed graphs resulting from test instances as inferring them from the grid representation is not always trivial. Figure 4 shows the `clingraph` output for the example environment from Figure 1.

[3] https://github.com/dbunker/railway/tree/main/graph

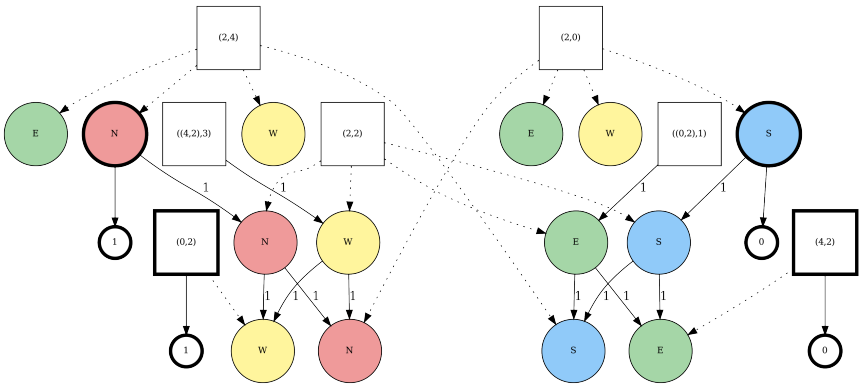


Fig. 4: Clingraph visualization of the condensed graph of example 1

The undirected base cells are represented here through white squares while their actual directed sub-nodes are represented through the connected colored circles. The numbers on the directed edges represent the length of the according edge. There are also certain nodes with a thicker outline. These nodes represent the train starting and end positions. Here the round directed nodes are always the start and the undirected square nodes the end positions. The small round nodes connected to them indicate which train ID starts or ends there.

This graph generation process has been consolidated to a bash script which can be run using `./show_graph.sh INSTANCE` in the repository. From there any Flatland environment instance can be provided and converted to a condensed graph representation which can then be visualized.

5.3 Flatdraw

Flatdraw is a tool combining two different functionalities regarding the creation of Flatland environments. It began as an idea to compress Flatland environments from their textual fact format into an image based representation.

5.3.1 Environment Compression

Image formats such as `png` are ideally suited for representing such environments as they all natively use the concept of a pixel raster which perfectly translates to the Flatland grid. In this way the positional information can be stored more efficiently as opposed to writing it out in a fact format. Another example benefit is that `png` images support native lossless compression methods that further reduce the size of the representations. As such, only track type information needs to be encoded in the color channel and can later use the images created by this to store the environments.

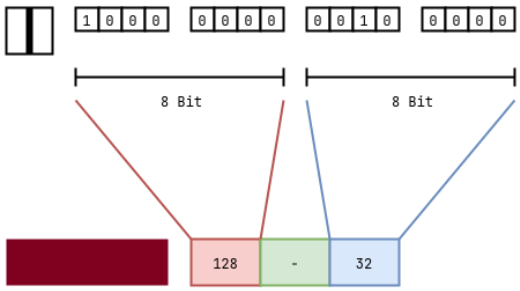


Fig. 5: Track type bitmap conversion to color value

Figure 5 illustrates how the track information is converted to a color value.

In this example, a horizontal straight track in its 16-bit transitional representation is split in half, yielding two 8-bit slices. As `png` images use a 4-byte `ABGR` representation for every pixel color, the first slice in the byte is the red channel and the second is the byte for the blue channel. This leaves room in the green and alpha bytes for additional information on train spawns, stations and more. Though in the implementation, only track information is retained as this tool was not the main focus of the project, and it is better to keep implementation simple.

When this conversion is applied to an arbitrary Flatland environment, an image representing all the environment's track information can be shown in a more optimized and space-efficient format. Figure 6 illustrates this, where a 16×16 Flatland environment is converted to its `png` representation, thereby reducing its size from about 4.5kB in the fact representation to only about $250B$ in the image representation.

For larger maps, especially sparse ones, this compression effect can become even more pronounced. Though as most state-of-the-art ASP solvers for this problem still have trouble solving for large environments the need for such a compression is probably not extremely high currently. The code for this compression implementation can be found in the `hweichelt/flatdraw`[4] GitHub repository.

5.3.2 Interactive Map Editor

This compression approach inspired a tool that makes creating these environments, in either the standard fact-format or the compressed image-format, easily accessible. This tool takes the form of a web-based interactive map editor where

[4] https://github.com/hweichelt/flatdraw

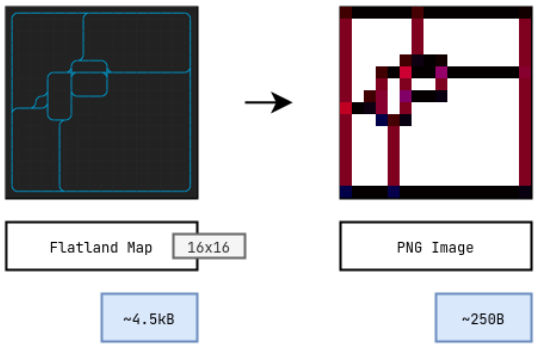


Fig. 6: Example conversion of a 16×16 environment

flatland environments can be drawn in a manner similar to paint. The implementation for this is provided in the `hweichelt/flatdraw`[4] repository. It is based on a locally run python `flask` server that uses `html` and `javascript` to provide the simple map editor UI as can be seen in Figure 7.

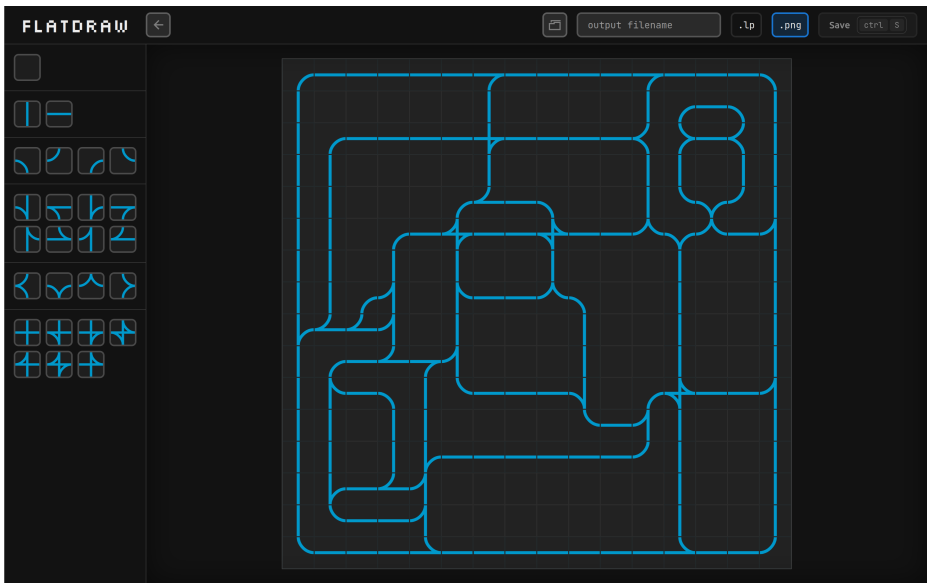


Fig. 7: Flatdraw UI

The editor is able to handle creating maps up to a size of 200×200 nodes and can open and save the environments as either `instance.lp` or `instance.png` files. In practice, the editor has proven useful as since the compressed graph benefits from large sparse environments and these can be very tedious to create by typing the facts by hand. The visual editor made it easy to create those maps while always being able to keep an overview of the current state of the environment. An example illustrating this is a map of the Berlin S-Bahn network, which was created using the tool[5].

6 Results

To evaluate this approach, an artificial set of benchmarks was created aimed at highlighting the differences between traditional time-based asp approaches and the order-based approach. The implementation was also tested on a set of shared common environments as well as randomly generated ones.

6.1 Benchmark Environments

A range of artificial benchmarking environments were created to evaluate the approach in specific scenarios. In Table 1 they are listed together with their defining properties. For this project, three example environments were also provided for testing purposes with the shared common environments. These can be found in Table 2 together with their properties.

To test how the Baseline and Ordering approaches function more broadly, they were run against a variety of randomly generated environments. These can be separated into four categories, "Sparse Few" representing a sparse simpler track environment with many trains, "Sparse Many" for sparse track and many trains, "Dense Few" for a denser more complex track environment with few trains, and "Dense Many" for dense environment with many trains. These variables were controlled through the parameters passed to the Flatland environment generator. A total of 15 environments were created for each category, each 35x35 with few trains being 4, many being 8 and complexity controlled via max rails between cities and max rail pairs in city parameters.

6.2 Benchmark Results

6.2.1 Artificial & Common Environments

The results of order-based and the baseline approach can be found in Tables 1, and 2. For computing these results, a system running clingo 5.7.1 was used on an Ubuntu 24.04 computer using an Intel i5 13600k CPU with 64GB of DDR5 RAM with 6000MT/s. The metric we are measuring and comparing here is the time it takes each approach to find any solution to the given environment.
The following is a reference for the column headings in Tables 1, and 2:

[5] https://github.com/hweichelt/flatdraw/tree/main/examples/berlin

- `x`: map width
- `y`: map height
- `s`: number of switches
- `r`: number of total rail nodes (switches included)
- `t`: number of trains
- `h`: time horizon

Benchmark Results (artifical)									
Environment	x	y	s	r	t	h	Baseline	Ordered	Speedup
crossing	3	3	1	5	2	10	0.010s	0.0045s	2.22
crossing_distractions	19	19	161	329	2	30	4.415s	0.1620s	27.25
crossing_grid	19	19	1	361	2	30	0.089s	0.1774s	0.50
crossing_large	19	19	1	37	2	30	0.068s	0.0188s	3.62
detour	16	16	2	254	2	100	2.647s	0.0912s	29.02
congestion	15	15	30	199	2	100	4.582s	0.0528s	86.78
congestion	15	15	30	199	4	100	14.124s	0.0929s	152.03
congestion	15	15	30	199	14	100	155.276s	0.7398s	209.89

Table 1: Benchmark results for artificial environments

Benchmark Results (common)									
Environment	x	y	s	r	t	h	Baseline	Ordered	Speedup
env_01	30	30	24	150	2	34	0.419s	0.0779s	5.38
env_02	30	30	42	203	2	98	5.414s	0.1142s	47.41
env_03	42	42	23	121	2	49	0.469s	0.0759s	6.18

Table 2: Benchmark results for common shared environments

6.2.2 Randomly Generated Environments

As can be seen in Table 3 showing the mean statistics associated with both methods, it is clear that the ordered encoding requires fewer grounded variables, but not on the order of number of time steps. It does, however have far fewer constraints, which would be expected due to the reduced time restrictions. The ordered encoding also has fewer choices than the baseline for few trains, but more for many trains, likely due to the trains' time nonrestrictive movement having a multiplicative effect with additional trains. It also has more conflicts, which would normally indicate a more constrained problem. The ordered encoding is much faster, likely primarily due to reduced constraints, though it may still be possible to add restrictions back without causing the constraints to increase as much.

7 Discussion

From the results of the benchmarks, the implemented approach is particularly more efficient in solving Flatland for larger environments with more trains.

Mean Statistics for Baseline Encoding					
Environment	**Variables**	**Constraints**	**Choices**	**Conflicts**	**Time**
SparseFew	83,697.00	7,015,958.00	8,352.33	45.78	7.308
SparseMany	261,710.50	31,595,730.00	13,930.00	52.50	15.558
DenseFew	167,351.83	22,915,630.00	8,868.08	45.17	27.136
DenseMany	444,018.80	88,625,030.00	33,047.00	163.00	76.277

Mean Statistics for Ordered Encoding					
Environment	**Variables**	**Constraints**	**Choices**	**Conflicts**	**Time**
SparseFew	50,381.00	129,670.10	606.07	153.21	0.133
SparseMany	135,166.40	431,970.80	36,995.73	164.64	0.425
DenseFew	112,450.67	298,096.80	1,105.33	397.83	0.292
DenseMany	315,452.67	1,003,585.00	46,721.75	214.00	1.081

Table 3: Benchmark statistics comparing Mean Baseline and Ordered strategies.

However, this comes with the loss of adhering to time constraints. Another restriction of this partial ordering approach is that it is only applicable to acyclic plans. This behavior is inherent to the approach, as was also mentioned in [4]. As such, instances that require cyclic train routes to satisfy all targets are not solvable with this approach. The encoding is also not currently able to handle two trains leaving from the same node at different times. This is due to removing time to generate the partial ordering which allows the trains to conflict. A potential solution is to create individual virtual incoming nodes for each train, resolving the overlapping conflict.

Although the implementation has these restrictions, it is able to benefit in scenarios where they can be loosened in favor of finding solutions faster. As seen in Tables 1, and 2 the approach is able to outperform the baseline in many scenarios sometimes up to two magnitudes faster in solving time.

Through the developed tooling around these approaches, the ASP flatland development environment was improved in regards to flatland map creation and representation (Flatdraw), interactive plan visualization (visualizer) and, specific to the approach, condensed graph visualization via Clingraph.

8 Future Work

Many ideas were approached as to how to conserve the time constraints of the original Flatland environments in the partially ordered solution. One idea to achieve this implicitly is through a global priority between the trains. Before the ordering is generated a score would be computed for each train that reflects the time difference between the train's earliest starting time and its latest arrival time. Then, while assigning orderings between trains, trains which have a lower score would be greater prioritized in these orderings than ones with a higher score. By this mechanism the trains that do not have much time to reach their goal would not have to wait for others as often and thus can reach their goal faster. This could be done using heuristics or optimization statements.

References

1. ABELS, D., JORDI, J., OSTROWSKI, M., SCHAUB, T., TOLETTI, A., WANKO, P.: Train scheduling with hybrid answer set programming. Theory and Practice of Logic Programming **21**(3), 317–347 (Apr 2020). https://doi.org/10.1017/s1471068420000046, http://dx.doi.org/10.1017/S1471068420000046
2. Danavulapadu, V.P., Singamsetty, P.: Trains scheduling problem with multiple lines. Scientific Reports **14**(1), 31129 (2024)
3. Hahn, S., Sabuncu, O., Schaub, T., Stolzmann, T.: Clingraph: Asp-based visualization. In: International conference on logic programming and nonmonotonic reasoning. pp. 401–414. Springer (2022)
4. Kaminski, R., Schaub, T., Son, T.C., Švancara, J., Wanko, P.: Routing and scheduling in answer set programming applied to multi-agent path finding: Preliminary report. arXiv preprint arXiv:2403.12153 (2024)
5. Laurent, F., Schneider, M., Scheller, C., Watson, J., Li, J., Chen, Z., Zheng, Y., Chan, S.H., Makhnev, K., Svidchenko, O., Egorov, V., Ivanov, D., Shpilman, A., Spirovska, E., Tanevski, O., Nikov, A., Grunder, R., Galevski, D., Mitrovski, J., Sartoretti, G., Luo, Z., Damani, M., Bhattacharya, N., Agarwal, S., Egli, A., Nygren, E., Mohanty, S.: Flatland competition 2020: Mapf and marl for efficient train coordination on a grid world (2021), https://arxiv.org/abs/2103.16511
6. Mohanty, S., Nygren, E., Laurent, F., Schneider, M., Scheller, C., Bhattacharya, N., Watson, J., Egli, A., Eichenberger, C., Baumberger, C., et al.: Flatland-rl: Multi-agent reinforcement learning on trains. arXiv preprint arXiv:2012.05893 (2020)
7. Perez, G., Glorian, G., Suijlen, W., Lallouet, A.: A constraint programming model for scheduling the unloading of trains in ports: Extended (2023), https://arxiv.org/abs/2312.13682
8. Scheffer, C.: Scheduling three trains is np-complete. In: Keil, J.M., Mondal, D. (eds.) Proceedings of the 32nd Canadian Conference on Computational Geometry, CCCG 2020, August 5-7, 2020, University of Saskatchewan, Saskatoon, Saskatchewan, Canada. pp. 87–93 (2020)